

INSTITUTO
FEDERAL
Santa Catarina

Resumo de matérias

Segurança da Informação

Eduardo Cardoso Oliveira, 2025

Sumário

1 Como estudar para esta disciplina? 5

2 Introdução à Segurança 5
2.1 O que é cibersegurança . . . 5
2.2 Segurança é sempre uma troca . . . 5
2.3 Vulnerabilidade . . . 6
2.4 Ameaça e ataque . . . 7
2.5 Propriedades básicas de segurança: a tríade CIA . . . 7
2.6 Classificação de ataques . . . 8

3 Introdução à Criptografia 8
3.1 Terminologia básica . . . 8
3.2 Criptografia clássica . . . 8
3.3 Criptografia moderna . . . 9
3.4 Algoritmo RSA e curva elíptica (ECC) . . . 9
3.5 Geração de números aleatórios e entropia . . . 10
3.6 Comunicação segura entre partes . . . 10

4 Função de Dispersão Criptográfica (Hash) 11
4.1 O que é uma função de hash . . . 11
4.2 HMAC . . . 12

5 Autenticação 12
5.1 Conceitos fundamentais . . . 12
5.2 Autenticação baseada em conhecimento: senhas . . . 12
5.2.1 Entropia de uma senha . . . 13
5.2.2 Armazenamento de senhas . . . 13
5.2.3 Principais ataques a senhas . . . 14
5.3 Autenticação baseada em posse . . . 14
5.3.1 Senhas de uso único: HOTP e TOTP . . . 14
5.3.2 Chaves de acesso (FIDO Passkeys) . . . 15
5.4 Autenticação entre serviços . . . 15
5.4.1 JSON Web Token (JWT) . . . 16

6 Certificado Digital e Assinatura Eletrônica 16
6.1 O problema da autenticidade da chave pública . . . 16
6.2 Certificado digital . . . 16
6.3 Infraestrutura de Chaves Públicas (ICP) . . . 16
6.4 Assinatura eletrônica . . . 17

7 Segurança em Redes de Computadores 18
7.1 VPN: Rede Virtual Privada . . . 18
7.2 Firewall . . . 18
7.3 Zona desmilitarizada (DMZ) . . . 18
7.4 Ataques que exploram vulnerabilidades de protocolos de rede . . . 19
7.5 Ataques de negação de serviço (DoS) . . . 19

8 Referências 20

Aviso importante e Isenção de responsabilidade

1. **Finalidade do documento**
   Este documento foi elaborado com o objetivo de servir como um material de apoio e revisão para os estudos das matérias do terceiro semestre de Análise e Desenvolvimento de Sistemas. Todos os conteúdos aqui presentes foram retirados de anotações pessoais e resumos, compilados para facilitar a compreensão do leitor.

2. **Fontes e Direitos Autorais**
   Os resumos e anotações aqui presentes foram desenvolvidos a partir dos materiais de sala disponibilizados pelos professores (como slides, apostilas, listas de exercícios, livros, etc), anotações de aula e pesquisas. Os direitos autorais sobre os materiais originais (aulas, livros, artigos) permanecem integralmente aos seus respectivos autores e editoras. Este documento é uma obra derivada, de caráter estudantil e sem fins lucrativos.

3. **Precisão das Informações**
   Desde já, o autor não oferece nenhuma garantia de que os conteúdos aqui contidos sejam precisos e corretos, por mais que todo esforço tenha sido feito para garantir que tais informações sejam corretas. Erros, omissões ou interpretações equivocadas podem existir. O autor não se responsabiliza por quaisquer consequências, acadêmicas ou de outra natureza, que possam surgir do uso deste material.

4. **Uso Recomendado**
   Este material **NÃO SUBSTITUI DE FORMA ALGUMA** a leitura de bibliografias recomendadas por professores, presença em aulas, consulta às fontes oficiais e, principalmente, as orientações do corpo docente. Utilize este documento como uma ferramenta complementar aos seus estudos.

5. **Vínculo Institucional**
   Este é um projeto independente e não possui qualquer vínculo oficial com o Instituto Federal de Santa Catarina Campus São José, a Coordenação do Curso de Análise e Desenvolvimento de Sistemas, ou os professores das disciplinas. Não há qualquer endosso ou revisão oficial do conteúdo por parte da instituição.

Apresentação do Material

Olá, colega!

Seja bem-vindo(a) ao meu resumo de estudos! Este documento foi criado ao longo do semestre com o objetivo de centralizar e facilitar o acesso aos conteúdos que foram abordados durante as aulas da disciplina de Segurança da Informação, ministrada pelo professor Emerson Ribeiro de Mello.

O que você vai encontrar neste documento?

- **Resumos:** Para esta disciplina trago resumo de todas as aulas, cobrindo desde os conceitos básicos de vulnerabilidade, ameaça e ataque, passando por criptografia, autenticação e certificados digitais, até segurança em redes de computadores.
- **Exemplos resolvidos:** Essa disciplina mistura teoria e matemática aplicada (entropia de senhas, cifras clássicas, HOTP/TOTP), então trago exemplos numéricos resolvidos sempre que possível.
- **Casos reais:** O professor traz bastante exemplo de incidentes reais de segurança (vazamentos, ataques, golpes), que considero essenciais para entender a aplicação prática dos conceitos, então procurei manter os principais no resumo.
- **Documentos e bibliografias:** Quando possível, vou disponibilizar materiais, resumos e referências para auxiliar.

Como este documento está organizado?

O material segue a mesma ordem das aulas do semestre: começamos com uma introdução aos conceitos básicos de segurança (vulnerabilidade, ameaça, ataque e a tríade CIA), seguimos para criptografia (clássica e moderna), depois autenticação (de usuários e entre sistemas), certificados digitais e assinatura eletrônica, e terminamos com segurança em redes de computadores.

Informações adicionais e contato

Este material foi feito de aluno para aluno; sugestões e colaborações são sempre bem-vindas! Se encontrar algum erro, tiver alguma sugestão para adicionar ao documento ou tirar alguma dúvida sobre qualquer matéria, fique à vontade para entrar em contato.

- **Autor:** Eduardo Cardoso Oliveira
- **E-mail Institucional:** eduardo.co22@aluno.ifsc.edu.br
- **Github:** https://github.com/EduCardoso22

Sumário

1 Como estudar para esta disciplina?

Segurança da Informação é uma disciplina que mistura teoria conceitual (o que é uma vulnerabilidade, o que é confidencialidade), matemática aplicada (entropia, criptografia) e muita discussão de casos reais. O professor traz constantemente exemplos de incidentes reais (vazamentos de dados, ataques famosos, golpes), e esses exemplos não são só "enfeite": eles ilustram diretamente os conceitos teóricos e ajudam a entender por que cada mecanismo de segurança existe. Por isso, meu principal conselho é prestar atenção nesses casos e tentar sempre conectá-los com a teoria (por exemplo: "por que o ataque ao C&M Software funcionou? Porque explorou o elo mais fraco, que são as pessoas").

Outro ponto importante é entender que boa parte da disciplina gira em torno de três propriedades básicas (confidencialidade, integridade e disponibilidade, a chamada tríade CIA) e de uma pergunta recorrente: "qual propriedade esse mecanismo garante, e qual ele não garante?"Praticamente todo mecanismo estudado (hash, salt, certificado digital, firewall, VPN) resolve um problema específico de uma dessas propriedades, mas não todos ao mesmo tempo. Ter esse "mapa mental"ajuda muito a não confundir, por exemplo, criptografia (que garante confidencialidade) com hash (que garante integridade). Por fim, pratique os cálculos matemáticos (entropia de senhas, cifra de César e Vigenère, HOTP/TOTP), pois eles aparecem com frequência nas avaliações.

2 Introdução à Segurança

2.1 O que é cibersegurança

Segurança computacional, cibersegurança ou segurança cibernética consiste em proteger sistemas de computadores, redes de computadores e dados contra ataques, danos ou acessos não autorizados, de forma a garantir a confidencialidade, a integridade e a disponibilidade dos sistemas e dados.

Um exercício interessante trazido pelo professor para ilustrar esse conceito é pensar em fechaduras físicas: entre duas fechaduras, qual é menos suscetível de ser aberta por uma pessoa não autorizada? Uma fechadura simples pode ser aberta por qualquer pessoa com um pouco de habilidade, ferramentas e tempo, sem precisar nem mesmo de acesso físico ao local (pensando em uma analogia com sistemas remotos); já uma fechadura mais robusta exige habilidade, ferramentas, tempo **e** acesso físico. Esse raciocínio ilustra bem que segurança nunca é "tudo ou nada": é sempre uma questão de dificultar o acesso não autorizado, elevando o custo (em tempo, dinheiro e habilidade) para um atacante.

2.2 Segurança é sempre uma troca

Segundo Bruce Schneier (2020), segurança é sempre uma troca (*trade-off*): segurança contra funcionalidades, segurança contra conveniência (usabilidade), segurança contra desempenho e segurança contra custo. Isso explica por que, muitas vezes, mecanismos de segurança mais robustos não são adotados: eles atrapalham a experiência do usuário ou custam caro demais para o benefício que trazem.

Um exemplo marcante trazido em aula para mostrar como negligenciar segurança pode ser grave: a vulnerabilidade da fechadura "Nexx Smart Wi-Fi Garage Door", em que todos os dispositivos compartilhavam a mesma credencial de acesso aos servidores da fabricante, e os dados pessoais do cliente e os comandos para abrir ou fechar o portão eram

enviados sem qualquer criptografia. Um pesquisador entrou em contato com a empresa em janeiro de 2023 e ficou mais de quatro meses sem resposta.

2.3 Vulnerabilidade

Uma **vulnerabilidade** é uma fraqueza do sistema, que pode ser em termos de procedimentos, configuração ou implementação, que pode ser explorada por um atacante para violar a segurança do sistema. Vulnerabilidades existem tanto em software quanto em hardware, podem ser descobertas facilmente ou demorar anos para serem descobertas, e podem ser corrigidas facilmente ou ser praticamente impossíveis de corrigir.

Uma **vulnerabilidade de dia zero** (*zero-day*) é aquela que ainda não é conhecida pelo fabricante do software ou hardware, ou que ainda não foi corrigida por ele. São as mais críticas e perigosas, pois os usuários não têm como se proteger, e costumam ser vendidas e usadas em ataques direcionados. Um exemplo clássico de vulnerabilidade de hardware são o **Spectre** e o **Meltdown** (2017), falhas de projeto em processadores Intel x86 produzidos depois de 1995 (que também afetam alguns processadores AMD e ARM), que demoraram dez anos para serem descobertas e permitem que um processo malicioso leia a memória de outros processos, sem explorar nenhuma vulnerabilidade de software.

O tempo entre a descoberta e a correção de uma vulnerabilidade também importa bastante: o ataque **WannaCry** (2017) explorou uma vulnerabilidade já corrigida pela Microsoft, mas muitos sistemas não haviam sido atualizados a tempo. Para catalogar vulnerabilidades publicamente conhecidas, existe o **CVE** (Common Vulnerabilities and Exposures), um sistema internacional que atribui um identificador único a cada vulnerabilidade (como `CVE-2023-12345`); já o **CWE** (Common Weakness Enumeration) é um catálogo padronizado de fraquezas de software e hardware que podem se tornar vulnerabilidades, como a validação insuficiente de entrada em páginas web (CWE-20).

Exemplo de fraqueza: um código Java que monta uma consulta SQL concatenando diretamente o valor recebido do usuário é vulnerável a *SQL Injection*:

```
// Vulneravel a SQL Injection
public void buscarUsuario(String id) throws SQLException {
    String query = "SELECT * FROM usuarios WHERE id = " + id;
    Statement stmt = conn.createStatement();
    ResultSet rs = stmt.executeQuery(query);
}

// Corrigido, usando parametros na consulta
public void buscarUsuario(String id) throws SQLException {
    String query = "SELECT * FROM usuarios WHERE id = ?";
    PreparedStatement pstmt = conn.prepareStatement(query);
    pstmt.setString(1, id);
    ResultSet rs = pstmt.executeQuery();
}
```

O princípio do **Security by Design** (segurança desde a concepção) prioriza a segurança desde o início do ciclo de vida do software, considerando ameaças, requisitos e controles já na fase de projeto, de forma que o sistema seja seguro por padrão, sem exigir ações adicionais de usuários ou administradores.

2.4 Ameaça e ataque

Uma **ameaça** é qualquer evento, ação, pessoa ou entidade que tem o potencial de causar danos a um sistema ou organização. Pode ser física (roubo, vandalismo, desastres naturais), lógica (malware, engenharia social), interna (funcionários desonestos) ou acidental (erros de configuração, falhas de hardware). Já um **ataque** é a concretização de uma ameaça, explorando uma vulnerabilidade para causar danos a um sistema ou organização, podendo ser direcionado ou indiscriminado, de curto ou longo prazo.

Um ponto que considero importante destacar é que os ataques estão ficando cada vez mais fáceis, baratos e eficazes com o tempo, graças ao avanço computacional. O exemplo do padrão de criptografia DES ilustra bem isso: em 1976, especialistas estimaram que seria necessário US$ 20 milhões para quebrá-lo; em 1997, foi quebrado por menos de US$ 250 mil; em 2012, pesquisadores quebraram o DES em 2 dias usando uma máquina de US$ 10 mil; e hoje é possível quebrá-lo em minutos usando um computador pessoal comum.

Por fim, é importante destacar o papel das **pessoas** na cadeia de segurança: Kevin Mitnick, um dos hackers mais famosos, afirmou que a engenharia social é a forma mais eficaz de invasão. Um exemplo recente e grave disso foi o roubo de R$ 541 milhões da C&M Software em 2025: criminosos assediaram um analista de backend da empresa, pagaram R$ 15 mil pelo acesso ao sistema e, meses depois, conseguiram desviar o valor em apenas três horas. As pessoas são, de fato, o elo mais fraco da cadeia de segurança, e os atacantes exploram a confiança, a curiosidade e a falta de conhecimento para obter acesso a sistemas críticos.

2.5 Propriedades básicas de segurança: a tríade CIA

As três propriedades básicas de segurança são conhecidas pela sigla em inglês **CIA** (Confidentiality, Integrity, Availability):

- **Confidencialidade**: a informação só deve ser revelada para usuários autorizados, durante a transmissão, o processamento e o armazenamento. É garantida principalmente por criptografia e controle de acesso. É importante não confundir confidencialidade com **privacidade**: privacidade é a capacidade de controlar a informação sobre si mesmo, enquanto a confidencialidade é a capacidade de controlar a informação de forma geral.

- **Integridade**: somente partes autorizadas podem alterar a informação, o que também garante o não repúdio e a autenticidade da informação. Pode ser verificada por mecanismos como checksum e assinaturas digitais, que não impedem a alteração, mas permitem detectá-la.

- **Disponibilidade**: a informação deve estar disponível quando necessário. Pode ser garantida por redundância, backups e sistemas de energia ininterrupta.

É necessário garantir as três propriedades simultaneamente para que a informação seja considerada segura. Além delas, outras propriedades importantes são a **autenticidade** (garantir que a informação é proveniente de quem diz ser) e a **responsabilidade** (*accountability*, a capacidade de rastrear ações até um indivíduo ou processo).

2.6 Classificação de ataques

Os ataques podem ser classificados em quatro categorias, de acordo com a propriedade que comprometem:

- **Interceptação**: uma parte não autorizada obtém acesso a uma informação (compromete a confidencialidade).
- **Modificação**: uma parte não autorizada altera uma informação e a envia para o destinatário (compromete a integridade).
- **Personificação**: uma parte não autorizada envia uma informação como se fosse uma parte legítima (compromete a autenticidade).
- **Interrupção**: uma parte não autorizada interrompe a comunicação entre duas partes legítimas (compromete a disponibilidade).

Um **incidente de segurança** é um evento que compromete a confidencialidade, a integridade ou a disponibilidade de um sistema ou organização. Por exemplo: um vazamento de dados compromete a confidencialidade; a perda de dados compromete a integridade; e a indisponibilidade de um sistema compromete a disponibilidade.

3 Introdução à Criptografia

3.1 Terminologia básica

Em um sistema criptográfico, M é o conjunto de mensagens em texto claro, K é o conjunto de chaves, C é o conjunto de mensagens cifradas, E é o algoritmo para cifrar ($E : M \times K \to C$) e D é o algoritmo para decifrar ($D : C \times K \to M$). A **criptoanálise** é o conjunto de técnicas usadas para quebrar criptosistemas, com o objetivo de recuperar a mensagem original sem conhecer a chave ou o algoritmo de cifragem.

3.2 Criptografia clássica

Na antiguidade, a criptografia tinha como objetivo principal garantir a confidencialidade das mensagens. A chave era compartilhada entre o emissor e o receptor, e a segurança do sistema dependia do sigilo tanto da chave quanto do próprio algoritmo. Os principais tipos de cifra clássica são:

- **Cifras de substituição**: os símbolos do texto claro são substituídos por outros símbolos, podendo ser monoalfabéticas ou polialfabéticas.
- **Cifras de transposição**: os símbolos do texto claro são rearranjados (permutados), ou seja, a ordem dos símbolos é alterada, mas os próprios símbolos continuam os mesmos.

Cifra de César (exemplo de substituição monoalfabética): uma das técnicas mais simples, usada por Júlio César na Roma antiga, que desloca o alfabeto em n posições fixas:

$$E_n(x) = (x - n) \mod 26 \qquad D_n(x) = (x + n) \mod 26$$

Por exemplo, com uma chave de deslocamento de 3 posições para a esquerda, o texto claro "IFSC"se transforma no texto cifrado "FCPZ". Essa cifra é facilmente quebrada por força bruta (existem apenas 25 chaves possíveis) ou por análise de frequência de letras (em português, a letra mais frequente é a vogal "a", com 14,63% de ocorrência).

Cifra de Vigenère (exemplo de substituição polialfabética): usa uma palavra-chave para cifrar o texto em claro, repetindo a palavra-chave até ter o mesmo tamanho do texto, e aplicando a cifra de César letra a letra, usando a letra correspondente da palavra-chave como deslocamento. Por exemplo, com a chave "IFSC"aplicada ao texto "SEGURANCA ADS":

```
Palavra-chave: IFSCIFSCI FSC
Texto em claro: SEGURANCA ADS
Texto cifrado: AJYWZFFEI FVU
```

A **Enigma**, máquina eletromecânica com rotores usada pela Alemanha nazista na Segunda Guerra Mundial, é um exemplo famoso de cifra polialfabética mais sofisticada, que teve seu código quebrado por matemáticos poloneses em 1933 e, posteriormente, teve sua quebra acelerada por Alan Turing e sua equipe em 1940.

3.3 Criptografia moderna

A criptografia moderna só se tornou possível com o uso de computadores. Além da confidencialidade, ela também tem como objetivo garantir a autenticidade e a integridade das mensagens, sendo baseada em problemas matemáticos difíceis de resolver. Diferente da criptografia clássica, os algoritmos criptográficos modernos são públicos e amplamente estudados: a segurança não depende do sigilo do algoritmo, mas sim da dificuldade de resolver o problema matemático e do sigilo da chave. Existem dois tipos de criptosistema:

- **Criptosistema simétrico**: a mesma chave é usada para cifrar e decifrar a mensagem. É mais eficiente que o assimétrico, sendo útil para armazenamento seguro de dados e comunicação segura entre duas partes, desde que a chave seja compartilhada previamente.
- **Criptosistema assimétrico**: duas chaves diferentes são usadas, uma para cifrar e outra para decifrar. Suas principais aplicações são comunicação segura (e-mail seguro, redes sociais), assinatura eletrônica, certificados digitais e troca de chaves de sessão (já que a criptografia simétrica é mais eficiente, a assimétrica é frequentemente usada apenas para trocar a chave simétrica com segurança).

3.4 Algoritmo RSA e curva elíptica (ECC)

O **RSA** foi proposto em 1977 por Ron Rivest, Adi Shamir e Leonard Adleman, sendo um dos algoritmos assimétricos mais usados atualmente. Ele é baseado na dificuldade de fatorar o produto de dois números primos grandes: dados dois números primos p e q, calcular $n = p \times q$ é fácil, mas, dado apenas n, é extremamente difícil encontrar p e q. O tamanho mínimo de chave recomendado atualmente é de 2048 bits.

Já a **Curva Elíptica Criptográfica (ECC)** se baseia na dificuldade de resolver o problema do logaritmo discreto entre pontos em uma curva elíptica. A grande vantagem do ECC é que chaves bem menores conseguem fornecer o mesmo nível de segurança que chaves RSA muito maiores, sendo também mais resistente a ataques quânticos (o RSA é vulnerável a ataques baseados no algoritmo de Shor):

Segurança (bits)	Tamanho RSA (bits)	Tamanho ECC (bits)
80	1.024	160
112	2.048	224
128	3.072	256
192	7.680	384
256	15.360	521

3.5 Geração de números aleatórios e entropia

A segurança de um sistema criptográfico depende diretamente da qualidade dos números aleatórios usados, seja na geração de chaves, em *salts* para senhas com hash, ou em números de uso único (*nonce*). Existem três tipos de geradores:

- **PRNG** (Pseudo-Random Number Generator): números pseudoaleatórios gerados por algoritmos determinísticos, como o Gerador Congruencial Linear. Não são adequados quando se exige segurança criptográfica, pois, dada a mesma semente, sempre geram a mesma sequência.
- **CSPRNG** (Cryptographically Secure PRNG): números aleatórios criptograficamente seguros, gerados a partir de fontes de entropia de alta qualidade (ruído atmosférico ou térmico, interação do usuário, interrupções de hardware).
- **TRNG** (True Random Number Generator): números verdadeiramente aleatórios, gerados por hardware especializado, geralmente usado em aplicações de segurança crítica.

Um caso interessante trazido em aula que ilustra a importância de uma boa fonte de entropia: um usuário do gerenciador de senhas RoboForm perdeu a senha de sua carteira de criptomoedas (com 43,6 BTC), mas pesquisadores fizeram a engenharia reversa do software e descobriram que o gerador de números pseudoaleatórios usava a data e hora do computador do usuário como semente. Como o usuário lembrava vagamente do período em que a criou (2013), foi possível gerar todas as senhas candidatas daquele intervalo e recuperar a senha perdida.

3.6 Comunicação segura entre partes

A principal dificuldade da criptografia simétrica é como compartilhar a chave secreta de forma segura entre as partes. O **protocolo de acordo de chaves Diffie-Hellman** resolve esse problema, permitindo que duas partes cheguem a um segredo compartilhado mesmo trocando informações por um canal inseguro, sem nunca transmitir a chave em si.

Exemplo resolvido: Alice e Bob concordam publicamente com um número primo $p = 23$ e uma base $g = 5$. Cada um escolhe um segredo privado: o segredo de Alice é $a = 6$ e o de Bob é $b = 15$. Alice calcula $A = g^a \bmod p$ e envia A para Bob; Bob calcula $B = g^b \bmod p$ e envia B para Alice. Por fim, Alice calcula o segredo compartilhado como $s = B^a \bmod p$, e Bob calcula o mesmo segredo como $s = A^b \bmod p$, chegando ao mesmo valor sem que o segredo em si jamais tenha trafegado pela rede. Vale notar que o Diffie-Hellman, sozinho, ainda é suscetível a ataques do tipo Man-in-the-Middle (MITM), pois não garante a autenticidade de quem está do outro lado da troca.

Já a **criptografia assimétrica** resolve, ao mesmo tempo, o problema da confidencialidade e da autenticidade: cada parte possui um par de chaves (pública e privada), em que a chave pública é compartilhada com todos e a chave privada é mantida em segredo.

- Para garantir **confidencialidade**: Bob cifra a mensagem com a chave pública de Alice, e somente Alice consegue decifrá-la com sua chave privada.
- Para garantir **autenticidade**: Alice assina a mensagem com sua chave privada, e Bob verifica a assinatura usando a chave pública de Alice.
- Combinando as duas: Bob assina a mensagem com sua própria chave privada e cifra o resultado com a chave pública de Alice; Alice, por sua vez, decifra a mensagem com sua chave privada e verifica a assinatura com a chave pública de Bob.

A **criptografia fim a fim** (E2EE, *end-to-end encryption*) garante que o conteúdo só possa ser acessado pelo remetente e pelo destinatário, nem mesmo o provedor do serviço de comunicação consegue ler o conteúdo das mensagens.

4 Função de Dispersão Criptográfica (Hash)

4.1 O que é uma função de hash

Uma função de dispersão criptográfica (*cryptographic hash function*) mapeia uma entrada de tamanho arbitrário para uma saída de tamanho fixo, chamada de **hash** ou resumo criptográfico. É uma função unidirecional: não é possível recuperar a entrada original a partir do hash. As principais propriedades que uma boa função de hash deve ter são:

- **Determinismo**: a mesma entrada sempre produz o mesmo hash.
- **Resistência a pré-imagem**: dado um hash h, deve ser computacionalmente inviável encontrar uma entrada m tal que $h = hash(m)$.
- **Resistência a segunda pré-imagem**: dada uma entrada m_1, deve ser inviável encontrar uma entrada m_2 diferente tal que $hash(m_1) = hash(m_2)$.
- **Resistência a colisões**: deve ser inviável encontrar duas entradas quaisquer diferentes que produzam o mesmo hash.
- **Difusão**: uma pequena alteração na entrada deve produzir um hash completamente diferente.

Entre os principais algoritmos de hash estão o MD5 (128 bits, hoje inseguro, embora ainda usado na NF-e no Brasil), o SHA-1 (160 bits, inseguro, usado no Git e em torrents; a Google apresentou um ataque de colisão prático em 2017), o SHA-2 (de 224 a 512 bits, usado pelo Bitcoin com SHA-256) e o SHA-3 (256 e 512 bits, padrão atual do NIST).

As funções de hash têm diversas aplicações: autenticação de usuários (senhas não armazenadas em texto claro, mas como hash), autenticação de mensagens, verificação de integridade de arquivos (como imagens ISO), assinaturas digitais e prova de trabalho (mineração de Bitcoin, que consiste em achar um hash que começa com um número mínimo de zeros).

4.2 HMAC

Como uma função de hash comum garante integridade, mas não autenticidade (um adversário poderia gerar o hash de uma mensagem modificada e enviá-la como se fosse legítima), existe o **HMAC** (Hash-based Message Authentication Code), que combina uma chave secreta com uma função de resumo criptográfico. Apenas quem conhece a chave secreta consegue gerar ou verificar o HMAC corretamente. Seu funcionamento é:

1. Emissor e receptor compartilham previamente uma chave secreta K.
2. O emissor calcula o HMAC da mensagem M usando K e envia a mensagem junto com o HMAC.
3. O receptor, ao receber M e o HMAC, recalcula o HMAC usando a mesma chave K e compara com o valor recebido.
4. Se os valores coincidirem, a mensagem é considerada autêntica e íntegra.

Exemplo com OpenSSL: o emissor gera o HMAC-SHA256 de um arquivo usando uma chave secreta e o envia junto com o arquivo:

```
# Gerar HMAC-SHA256 de um arquivo usando a chave 'segredo'
openssl dgst -sha256 -hmac segredo contrato.pdf
# Saida: HMAC-SHA2-256(contrato.pdf)= bc252b0cd3da5a85...
```

O receptor pode repetir o mesmo comando com a mesma chave para verificar a autenticidade e a integridade do arquivo.

5 Autenticação

5.1 Conceitos fundamentais

Uma **política** define o que é permitido ou negado (diretrizes, regras, procedimentos), enquanto um **mecanismo** é como essa política é implementada, aplicada e executada. A **identidade digital** é a representação única de uma entidade usada para identificá-la em uma transação eletrônica, e a **autenticação** é o processo de verificar se a entidade realmente controla os autenticadores associados àquela identidade digital.

Os autenticadores são divididos em três **fatores de autenticação**: **conhecimento** (senha, PIN, algo que você sabe), **posse** (token, cartão, celular, algo que você tem) e **inerência** (características biométricas, algo que você é). A **autenticação multifator** (MFA) combina dois ou mais fatores de categorias distintas para tornar o processo mais robusto.

5.2 Autenticação baseada em conhecimento: senhas

A senha foi criada em 1961 por Fernando Corbató no MIT, para permitir que vários usuários acessassem o mesmo computador ao mesmo tempo. Suas vantagens são não exigir equipamento adicional e ser fácil de alterar; suas desvantagens são a vulnerabilidade a ataques de força bruta, phishing e observação por cima do ombro (*shoulder surfing*), além da dificuldade de serem memorizadas quando complexas.

5.2.1 Entropia de uma senha

A **entropia** é uma medida matemática que indica o grau de imprevisibilidade de uma senha, refletindo o número de possibilidades que um atacante precisaria testar até encontrar a senha correta. Quanto maior a entropia, mais difícil é quebrar a senha por força bruta. A fórmula da entropia teórica é:

$$E = \log_2(R^L) = L \times \log_2(R)$$

onde E é a entropia em bits, R é o número de caracteres possíveis no alfabeto usado e L é o comprimento da senha.

Exemplo resolvido: uma senha de 4 dígitos numéricos (apenas os caracteres de 0 a 9, logo $R = 10$) tem entropia $E = \log_2(10^4) = 13,29$ bits, o que significa cerca de $2^{13,29} \approx 10.000$ tentativas para descobri-la por força bruta. Já uma senha de 11 caracteres usando letras maiúsculas, minúsculas, números e símbolos ($R = 94$) tem entropia $E = \log_2(94^{11}) \approx 73,88$ bits, um espaço de busca muitíssimo maior.

É importante notar que o padrão "palavra do dicionário + número ou símbolo"tem uma entropia teórica enganosa: considerando um conjunto de 2048 palavras comuns usadas em 4 posições, a entropia seria de apenas $\log_2(2048^4) = 44$ bits, bem menor do que uma senha verdadeiramente aleatória de tamanho equivalente, pois as palavras que as pessoas escolhem são mais previsíveis do que caracteres aleatórios.

Segundo o NIST (Grassi; Garcia; Fenton, 2020, com revisão de 2024), uma boa política de senhas deve exigir no mínimo 8 caracteres (idealmente pelo menos 15), permitir pelo menos 64 caracteres, aceitar todos os caracteres ASCII e Unicode, **não** deve impor restrições de composição arbitrárias (como exigir misturar letras, números e símbolos) e **não** deve exigir troca periódica de senha, exceto quando houver evidência de comprometimento.

5.2.2 Armazenamento de senhas

Existem várias formas de armazenar a senha de um usuário em um sistema, da pior para a melhor prática:

- **Texto claro**: a senha fica visível para qualquer pessoa com acesso ao banco de dados, incluindo administradores. Prática fortemente desaconselhada.
- **Cifrada com algoritmo simétrico**: a senha é cifrada (por exemplo, com AES) antes de ser armazenada. Esse método permite recuperar a senha original com a chave, o que atualmente também não é recomendado.
- **Hash**: a senha é armazenada como um resumo criptográfico unidirecional. O problema é que, se dois usuários escolherem a mesma senha, o hash resultante será idêntico, o que facilita ataques com tabelas pré-computadas.
- **Hash combinado com salt**: o **salt** é uma sequência aleatória concatenada à senha antes de calcular o hash ($hash$ = função_hash($senha \parallel salt$)), e precisa ser armazenado junto com o hash. Assim, mesmo que dois usuários escolham a mesma senha, seus hashes serão diferentes.

Como funções de dispersão criptográfica comuns (MD5, SHA-1, SHA-256) são projetadas para serem **rápidas**, o que as torna vulneráveis a ataques de força bruta, para

armazenar senhas o ideal é usar funções de hash propositalmente **lentas**, que consomem bastante CPU ou memória, como bcrypt, scrypt, PBKDF2 ou Argon2 (este último venceu a Password Hashing Competition).

Uma prática ainda mais recomendada é combinar **salt** e **pepper**: enquanto o salt evita que senhas iguais tenham o mesmo hash, ele não impede, sozinho, ataques de força bruta contra uma base de senhas comprometida (já que o atacante tem acesso ao salt). O **pepper** é um valor aleatório e secreto, concatenado à senha antes do cálculo do hash, mas armazenado separadamente do hash e do salt (por exemplo, em um cofre de senhas), de forma que, mesmo que a base de dados seja comprometida, o pepper não fique disponível para o atacante.

5.2.3 Principais ataques a senhas

- **Força bruta**: o atacante tenta todas as combinações possíveis de senha. Uma senha de 9 caracteres alfanuméricos tem cerca de $1,35 \times 10^{16}$ combinações; testando 100 mil senhas por segundo, levaria cerca de 4 anos para ser descoberta, mas usando heurísticas sobre o comportamento humano (como o padrão "Nome+Ano") esse tempo pode cair para minutos.
- **Dicionário**: o atacante tenta senhas comuns ou palavras de dicionário (nomes, datas de nascimento, placas de carro), aproveitando que a maioria das pessoas escolhe senhas previsíveis.
- **Tabela arco-íris** (*rainbow table*): tabelas de hash pré-computadas, usadas para descobrir senhas candidatas muito mais rapidamente do que calculando o hash de cada tentativa. O uso de salt (combinado ou não com pepper) e de algoritmos de hash lentos é suficiente para se proteger desse tipo de ataque.

Para aumentar a segurança do processo de autenticação baseada em senha, pode-se usar: bloqueio de contas após várias tentativas falhas, tempo de espera entre tentativas, CAPTCHA (para evitar ataques automatizados) e, principalmente, autenticação multifator.

5.3 Autenticação baseada em posse

Inclui certificados digitais armazenados em cartão inteligente ou token (como o e-CPF e o e-CNPJ), senhas de uso único (token físico, aplicativo ou SMS) e chaves de segurança de hardware conectadas ao computador (como YubiKeys compatíveis com FIDO2).

5.3.1 Senhas de uso único: HOTP e TOTP

O **HOTP** (HMAC-based One-Time Password) aplica HMAC sobre um segredo compartilhado K (uma chave secreta de 128 bits) e um contador C (um número inteiro incrementado a cada autenticação, por ambas as partes):

$$HOTP(K, C) = truncate(HMAC_{SHA1}(K, C))$$

Já o **TOTP** (Time-based One-Time Password) aplica o mesmo princípio do HOTP, mas usando o tempo como contador: em vez de um contador incrementado manualmente,

usa o número de segundos desde 1º de janeiro de 1970 (época Unix), dividido por um intervalo de tempo fixo (geralmente 30 segundos):

$$TOTP(K) = HOTP\left(K, \left\lfloor \frac{T}{intervalo} \right\rfloor\right)$$

O TOTP é preferível ao HOTP por ser mais seguro, já que o curto tempo de vida da senha (30 segundos) limita bastante a janela de oportunidade de um atacante. O segredo precisa ser armazenado com segurança em ambos os lados: se for perdido por qualquer uma das partes, a autenticação deixa de funcionar.

É importante notar, porém, que OTP **não** é imune a ataques do tipo Man-in-the-Middle: ferramentas como o Evilginx conseguem atuar como um proxy transparente entre o usuário e o serviço legítimo, capturando tanto as credenciais quanto o cookie de sessão gerado após a autenticação de dois fatores, contornando efetivamente a proteção do MFA.

5.3.2 Chaves de acesso (FIDO Passkeys)

As **chaves de acesso** (*passkeys*), baseadas no padrão FIDO, são usadas como segundo fator ou como único fator (autenticação sem senha, *passwordless*), permitindo inclusive autenticação sem nome de usuário (*usernameless*) e autenticação cruzada (por exemplo, usar a passkey do celular para autenticar no computador). Os autenticadores FIDO podem ser dispositivos (celular, computador) ou tokens físicos, e as chaves de acesso são sincronizadas automaticamente entre os dispositivos de um mesmo ecossistema (como o ecossistema Google ou Apple).

Comparando os métodos de posse: um certificado digital é mais seguro que a combinação senha + OTP, mas é mais difícil de usar, especialmente em dispositivos móveis. O OTP está suscetível a ataques de phishing, malware no dispositivo do usuário e MITM. Já as chaves de acesso FIDO são, ao mesmo tempo, mais seguras e mais fáceis de usar, sendo imunes a ataques do tipo MITM.

5.4 Autenticação entre serviços

Quando dois sistemas (e não pessoas) precisam se autenticar mutuamente, como em uma API, existem alguns mecanismos comuns:

- **Protocolo de desafio e resposta**: cliente e servidor compartilham previamente um segredo. O servidor gera um desafio (uma mensagem aleatória e única, o *nonce*) e o cliente responde combinando o segredo compartilhado com o desafio recebido, geralmente usando HMAC.
- **Autenticação básica HTTP** (*Basic Auth*): o servidor avisa ao cliente que a autenticação é necessária com o código `401 Unauthorized`, e o cliente então envia as credenciais codificadas em Base64 no cabeçalho `Authorization`. Como a codificação Base64 **não** é criptografia (apenas uma forma de representar dados binários em ASCII), esse método só é seguro quando usado em conjunto com HTTPS.
- **Autenticação por token ou API Key**: uma cadeia de caracteres gerada aleatoriamente, geralmente opaca (só quem a gerou consegue decodificá-la), usada para autenticar o cliente a cada requisição. Pode ser enviada no cabeçalho HTTP (o mais comum é usar `Authorization: Bearer <token>`), como parte da query string da URL, ou como um cookie. Tokens nunca devem ser deixados no código-fonte e só devem ser enviados sobre HTTPS.

5.4.1 JSON Web Token (JWT)

O **JWT** é um padrão aberto que define um formato compacto para troca segura de informações entre partes, como um objeto JSON. É amplamente usado para autenticação e autorização em APIs RESTful, sendo assinado digitalmente para garantir integridade e autenticidade (podendo também ser criptografado para garantir confidencialidade). Um JWT é composto por três partes, separadas por pontos e codificadas em Base64URL: `header.payload.signature`, em que o *header* contém o tipo do token e o algoritmo de assinatura, o *payload* contém as declarações (*claims*, como dados do usuário) e a *signature* é a assinatura do token, calculada sobre o header e o payload usando a chave secreta.

6 Certificado Digital e Assinatura Eletrônica

6.1 O problema da autenticidade da chave pública

A criptografia de chave pública resolve boa parte dos problemas de confidencialidade e integridade, mas deixa uma lacuna importante: como garantir que uma determinada chave pública realmente pertence à pessoa ou ao site que diz ser o dono dela? Se um atacante conseguir fazer o papel de homem do meio (MITM) durante a troca inicial de chaves públicas, ele pode se passar por qualquer uma das partes sem que ninguém perceba.

6.2 Certificado digital

Um **certificado digital** (ou certificado de identidade) associa os dados de uma entidade (pessoa física, pessoa jurídica ou servidor) a uma chave pública, e possui validade (não é válido antes de uma data nem depois de outra) e pode ser revogado. A confiança no certificado vem da confiança no seu emissor, de forma parecida com a confiança que depositamos em um cartório para reconhecer firma.

Quem emite certificados é a **Autoridade Certificadora (AC)**, uma entidade considerada uma terceira parte confiável: duas partes que não confiam uma na outra podem confiar em uma AC comum. A confiança em uma AC é baseada na sua reputação, nas suas políticas de emissão de certificados e nos procedimentos de validação da identidade do solicitante. Navegadores e sistemas operacionais mantêm uma lista pré-instalada de ACs confiáveis (como a Let's Encrypt, que emite certificados gratuitos para servidores HTTPS, válidos por 90 dias e renovados automaticamente).

Um **certificado autoassinado** é emitido e assinado pela própria entidade, sem passar por nenhuma AC. É útil para testes e desenvolvimento, mas nunca deve ser usado em produção: navegadores e sistemas operacionais não confiam nele, o cliente é alertado de que o certificado não é confiável, e ele permite comunicação cifrada, mas **não** garante a autenticidade do servidor, ficando suscetível a ataques MITM e de personificação.

6.3 Infraestrutura de Chaves Públicas (ICP)

A **ICP** é o conjunto de entidades, políticas, procedimentos e tecnologias que viabilizam a emissão e o gerenciamento de certificados digitais, formando uma hierarquia de confiança:

- **Autoridades Certificadoras (AC)**: responsáveis por emitir os certificados digitais para as entidades.

- **Autoridades de Registro (AR)**: responsáveis por verificar a identidade de quem solicita o certificado.
- **Autoridades de Carimbo do Tempo (ACT)**: responsáveis por atestar a data e a hora em que uma mensagem foi assinada.
- **Lista de Certificados Revogados (CRL)**: usada para verificar se um certificado já foi revogado antes de confiar nele.

Uma **AC Raiz** emite certificados para **ACs intermediárias**, que por sua vez emitem certificados para as entidades finais, formando uma cadeia de confiança. Uma ICP pode ser **pública** (controlada por uma entidade pública ou privada reconhecida amplamente, como a ICP-Brasil, Let's Encrypt ou DigiCert) ou **privada** (controlada pela própria organização, dando controle total sobre a identidade e a verificação de credenciais, sendo útil, por exemplo, para dispositivos IoT ou servidores com nome de domínio interno).

A **Infraestrutura de Chaves Públicas Brasileira (ICP-Brasil)**, regulamentada pela Medida Provisória nº 2.200-2/2001, é a cadeia hierárquica de confiança que viabiliza a emissão de certificados digitais para identificação virtual de cidadãos e empresas no Brasil, sendo responsável pela emissão do e-CPF e do e-CNPJ. Existem dois tipos principais: o **A1** (arquivo armazenado no computador, válido por 1 ano) e o **A3** (armazenado em cartão, token criptográfico ou em nuvem via HSM remoto, válido por 3 anos).

Centralizar a confiança em ACs também traz riscos: se uma AC for comprometida, todos os certificados emitidos por ela ficam comprometidos. Em 2011, a AC DigiNotar foi comprometida, e certificados falsos foram usados para realizar ataques MITM contra usuários iranianos nos domínios da Google.

6.4 Assinatura eletrônica

A **assinatura digital**, usando criptografia assimétrica, permite verificar a autoria e a integridade de um documento: aplica-se uma função de resumo criptográfico sobre o documento, e o resultado é assinado com a chave privada do signatário. Ela fornece o chamado **não repúdio**: o signatário não pode negar que assinou a mensagem, já que apenas ele detém sua chave privada.

No Brasil, a Lei nº 14.063/2020 classifica a assinatura eletrônica em três níveis, de acordo com o grau de confiabilidade:

- **Assinatura eletrônica simples**: apenas permite identificar o signatário e associa dados a outros dados em formato eletrônico.
- **Assinatura eletrônica avançada**: usa certificados digitais não emitidos necessariamente pela ICP-Brasil, ou outro meio de comprovação de autoria aceito pelas partes envolvidas. A assinatura GOV.BR é um exemplo desse tipo.
- **Assinatura eletrônica qualificada**: usa certificado digital emitido no âmbito da ICP-Brasil, sendo o nível mais elevado de confiabilidade, e tem a mesma validade jurídica de uma assinatura física.

7 Segurança em Redes de Computadores

7.1 VPN: Rede Virtual Privada

Uma **VPN** (Virtual Private Network) é uma rede privada que utiliza a infraestrutura de uma rede pública (normalmente a Internet) para estabelecer conexões seguras, por meio de túneis criptografados, entre dispositivos. Existem três tipos principais: **VPN de acesso remoto** (usuários acessam a rede corporativa remotamente, como se estivessem fisicamente no local), **VPN de site a site** (filiais de uma organização se conectam de forma segura, compartilhando recursos como se estivessem na mesma rede) e **VPN pessoal** (protege a navegação em redes públicas, embora o operador da VPN consiga, em tese, monitorar o tráfego, e a velocidade da conexão costuma ser reduzida).

7.2 Firewall

Um **firewall** é um ponto de controle que mantém acessos não autorizados fora do perímetro de segurança, ao mesmo tempo em que possibilita o acesso autorizado a sistemas externos, evitando que vulnerabilidades de serviços internos sejam exploradas. Pode ser de **máquina** (perímetro de segurança é a própria máquina, analisando todo o tráfego de entrada e saída) ou de **rede** (perímetro de segurança é toda a rede local da organização, obrigando todo o tráfego entre a rede interna e externa a passar por ele).

O firewall de rede centraliza o controle de acesso (protegendo mesmo máquinas de usuários que não estejam totalmente seguras) e é o local ideal para aplicar a tradução de endereços de rede (NAT). Porém, ele **não** protege contra ataques vindos de dentro da própria rede interna, e por isso é recomendado combiná-lo com firewalls pessoais em cada máquina.

Os principais tipos de firewall, da camada mais baixa para a mais alta, são:

- **Filtro de pacotes**: atua na camada de rede, analisando apenas cabeçalhos (endereço IP de origem/destino, porta de origem/destino, interface, protocolo), sem se preocupar com o conteúdo dos pacotes.
- **Inspeção de estado**: atua na camada de transporte, mantendo o estado de todas as conexões que passam por ele, conseguindo determinar se um pacote faz parte de uma conexão existente ou de uma nova conexão, analisando as flags TCP (SYN, ACK, etc).
- **Gateway no nível de aplicação**: atua na camada de aplicação, analisando o conteúdo dos pacotes e permitindo um controle bem mais detalhado do tráfego (como bloquear o download de arquivos `.exe`), mas exigindo mais processamento.
- **Web Application Firewall (WAF)**: especializado em proteger aplicações web, bloqueando ataques como SQL Injection, Cross-Site Scripting (XSS) e Cross-Site Request Forgery (CSRF). Serviços de CDN como Cloudflare e AWS WAF oferecem esse tipo de proteção com regras atualizadas.

7.3 Zona desmilitarizada (DMZ)

A **DMZ** (Demilitarized Zone) é um segmento de rede isolado entre a rede interna e a Internet, funcionando como uma ”zona neutra”. Ela protege a rede interna caso um

servidor acessível externamente (como um servidor web, DNS ou SMTP) seja comprometido, já que esses serviços ficam fisicamente separados da rede corporativa interna. Os servidores na DMZ precisam seguir rigorosamente as políticas de segurança e estar sempre atualizados, justamente por ficarem mais expostos.

7.4 Ataques que exploram vulnerabilidades de protocolos de rede

A Internet foi projetada para ser aberta, robusta e escalável, mas **não** foi projetada, originalmente, para ser segura: no final da década de 1990, o próprio IETF indicava que a segurança seria responsabilidade dos usuários finais, e não da rede em si (o chamado modelo de segurança fim a fim).

- **Sniffing**: interceptação e análise de dados transmitidos pela rede (Ethernet, Wi-Fi, Bluetooth). O atacante precisa estar na mesma rede da vítima ou controlar um roteador intermediário. É um ataque passivo, difícil de detectar; a principal defesa é usar criptografia (VPN, HTTPS, SSH, E2EE).
- **Spoofing**: falsificação de identidade para enganar dispositivos ou usuários da rede, sendo um ataque ativo. No **ARP spoofing**, o atacante descobre o endereço MAC do gateway e da vítima, e então envia respostas ARP falsas para atualizar a tabela ARP (cache) da vítima, redirecionando o tráfego para si. No **IP spoofing**, o atacante falsifica o endereço IP de origem para mascarar sua identidade.
- **DNS poisoning (cache poisoning)**: corrompe o cache de um servidor DNS, inserindo registros falsos, o que permite ao atacante redirecionar o tráfego para servidores maliciosos. O **DNSSEC** usa criptografia de chave pública para garantir a autenticidade e a integridade dos registros DNS, protegendo contra esse tipo de ataque.
- **BGP hijacking** (sequestro de BGP): como o BGP (Border Gateway Protocol) é o protocolo de roteamento entre sistemas autônomos na Internet, um ataque desse tipo ocorre quando um sistema autônomo anuncia rotas falsas para um bloco de IP, desviando o tráfego para si ou para servidores maliciosos. Exemplos reais incluem a China, que em 2010 chegou a anunciar rotas para 15% do tráfego global por 18 minutos.

7.5 Ataques de negação de serviço (DoS)

Um **ataque de negação de serviço (DoS)** tem como objetivo sobrecarregar um servidor ou serviço, tornando-o indisponível para usuários legítimos, muitas vezes combinado com IP spoofing para dificultar a identificação do atacante. Alguns tipos vistos em aula, organizados pela camada da pilha de protocolos que exploram:

- **IP spoofing** (camada de rede): o atacante forja o endereço IP de origem dos pacotes.
- **SYN flood** (camada de transporte): o atacante envia um grande volume de segmentos TCP SYN com IP de origem forjado, mas nunca responde ao SYN-ACK do servidor com o ACK final, fazendo com que a vítima mantenha diversas conexões

"meio-abertas"até o tempo limite (timeout) expirar. É considerado um ataque volumétrico.

- **Slowloris** (camada de aplicação): tem como alvo servidores web baseados em conexões persistentes (como o Apache HTTP), e tem a vantagem, do ponto de vista do atacante, de não exigir muito recurso de rede nem de processamento para ser executado.

8 Referências

Referências

[1] GOOGLE DRIVE. Segurança da Informação. Disponível em: `https://drive.google.com/drive/folders/1EiEEEoN1t6jj7QoEIIpj8MOOzsTqRw00?usp=drive_link`. Acesso em: 30 out. 2026.

[2] MELLO, Emerson Ribeiro de. Material de aula — Segurança da Informação. Instituto Federal de Santa Catarina, Campus São José, 2025.

[3] SANTOS BARRETO, Jeanine dos et al. *Fundamentos de Segurança da Informação*. 1. ed. Porto Alegre: Sagah, 2018. ISBN 9788595025875.

[4] STALLINGS, William. *Criptografia e Segurança de Redes: Princípios e Práticas*. 4. ed. São Paulo: Pearson Prentice Hall, 2007.

[5] SCHNEIER, Bruce. *Clique aqui para matar todo mundo*. 2020.

[6] GRASSI, Paul; GARCIA, Michael; FENTON, James. *NIST Special Publication 800-63-3: Digital Identity Guidelines*. 2020.

[7] BISHOP, Matt. *Computer Security: Art and Science*. 1. ed. Addison-Wesley, 2003. ISBN 0201440997.